RESEARCH daita_test

RESEARCH daita_test

Scope: (1) DAITA / maybenot traffic-analysis defense — current state of the maybenot framework and Mullvad's DAITA v1/v2; (2) VPN testing methodology — Linux netns rigs, nftables DPI/leak blocking, tc netem impairment, iperf3 throughput, DNS / kill-switch leak testing. Access date for all sources: **2026-06-25**.

---

1. maybenot framework — current state (2025-2026)

What it is. maybenot is an open-source **framework for traffic-analysis defenses** that "hide patterns in encrypted communication" to increase the uncertainty of a network attacker. It is an evolution/generalization of the Tor **Circuit Padding Framework** (Perry & Kadianakis), generalized to support many protocols (TLS, QUIC, **WireGuard**, Tor). Originally published as an academic paper (Pulls, WPES'23 / arXiv 2304.09510). [maybenot GitHub], [arXiv 2304.09510], [ACM WPES'23 DOI 10.1145/3603216.3624953].

Model — probabilistic state machines. A defense is expressed as **a list of probabilistic state machines** plus **limits on padding and blocking actions**. Each machine reacts to events (packet sent/received, timers) and triggers two classes of action:

- **Padding** — inject cover/dummy packets.
- **Blocking** — delay/hold outgoing traffic (timing normalization). The "limits" cap how much padding traffic and how much outgoing-traffic blocking a machine may impose (the **padding budget**). [maybenot GitHub], [pulls.name DAITA blog].

Implementation / crates (Rust). Written in Rust, **MSRV 1.85**. The workspace publishes several crates:

- `maybenot` — core framework. Latest version **2.2.2**. [libraries.io/cargo/maybenot]
- `maybenot-simulator` — testing/eval simulator. Latest **2.2.1**. [crates.io maybenot-simulator]
- `maybenot-ffi` — C FFI wrapper (used for non-Rust integration).
- `maybenot-machines` — a published library of useful/ready-made machines, latest **1.0.1**. [search result, crates.io maybenot-machines] So the **2.x line** is current (core 2.2.2). [maybenot GitHub], [libraries.io].

Integration. maybenot "is used by Mullvad VPN in DAITA," via a **WireGuard-Go integration**. Third parties are adopting it: NetBird has an open issue to add DAITA via the maybenot framework. [maybenot GitHub], [netbirdio/netbird#2366].

---

2. Mullvad DAITA — the three defenses (the Mullvad-parity target)

DAITA = **Defense Against AI-guided Traffic Analysis**, built on maybenot, developed with Karlstad University. Resists ML/AI **website-fingerprinting** attacks (Deep Fingerprinting "DF", Robust Fingerprinting "RF"). Three mechanisms (Mullvad's own framing): [mullvad.net/vpn/daita], [mullvad.net DAITA blog].

1. **Packet Size Normalization (constant packet size).** All packets sent over the VPN are padded to **one constant size**. Small packets are especially revealing, so this removes size as a feature.
2. **Dummy Packet Injection (cover traffic).** Dummy packets are interspersed **unpredictably** to mask routine signals — observer can't separate real activity from background noise.
3. **Traffic Pattern Distortion.** During bursty activity (e.g. a page load) DAITA sends cover traffic **in both directions** (client⍰relay) to distort the recognizable shape of a site visit.

---

3. DAITA v1 — hardcoded client machines (concrete numbers)

In **v1** the **client** runs **four hardcoded state machines**: [pulls.name 2025-03-27 DAITA v1 and v2 blog], [WebSearch summary].

1. **Inactivity padding** — sends a padding packet if **no data has been sent on the tunnel for a randomized [1.5, 9.5] seconds**.
2. **3rd-packet ratio** — ensures a packet is sent **for every 3rd packet received**.
3. **5th-packet ratio** — ensures a packet is sent **for every 5th packet received**.
4. **Randomized padding** — sends padding after randomized delays, active when the connection is otherwise idle.

Relay-side (v1): each relay defense = **three machines** — a **NetFlow** machine, an **Interspace-inspired padding** machine, and **a unique machine generated to reduce accuracy against DF and RF**. [pulls.name DAITA blog].

---

4. DAITA v2 — dynamic negotiated defenses (the current generation)

v2 overhauls the architecture: **no more hardcoded client machines — defenses are dynamic and negotiated.** [pulls.name DAITA blog], [cyberinsider DAITA v2], [mullvad blogs].

- **Negotiation.** When a v2 client negotiates DAITA, the **relay selects from a database of defenses** and **sends the appropriate machines + limits (padding budgets) to the client**. Strategy is chosen server-side and pushed.
- **Bandwidth overhead.** v2 uses roughly **half as many dummy packets** as v1 ("the same level of protection at **half the average bandwidth overhead**"), via more precise insertion logic — notably better on mobile. [pulls.name], [cyberinsider].
- **Protection retained.** Internal benchmarks: v2 maintains v1-level protection against **Deep Fingerprinting** and **Robust Fingerprinting** while cutting average bandwidth. [cyberinsider], [pulls.name].
- **Platform availability.** DAITA is shipped on Mullvad clients incl. Linux and macOS (Mullvad blog "DAITA now available on Linux and macOS"). [mullvad blog].

Parity design takeaways for a self-hosted VPN: implement (a) constant packet padding to a single MTU-bounded size, (b) probabilistic dummy injection with a capped padding budget, (c) bidirectional cover traffic during bursts, and (d) server-pushed machine selection (v2 model) rather than hardcoded client timers. Reuse the **maybenot** crate (core 2.2.2) + `maybenot-machines` rather than reimplementing the state-machine engine.

---

5. VPN testing methodology

5.1 Linux network namespaces (netns) — isolated test rig

- `ip netns` creates isolated network stacks; pair namespaces with **veth** virtual-ethernet links to build a client-ns ? gateway/relay-ns ? "internet-ns" topology without touching host/production traffic. [oneuptime ip netns guide], [girondi.net network namespaces].
- Run the VPN client in one ns, the relay/server in another, and a synthetic "internet" upstream in a third — fully reproducible, CI-friendly, no real WAN. [girondi.net], [oneuptime].

5.2 tc netem — network impairment emulation

- `netem` is a **tc qdisc** that injects **latency, jitter, packet loss, duplication, reordering, corruption**. Apply it **inside a namespace** on the veth so only the test path is impaired. [man7 tc-netem(8)], [oneuptime netem RHEL9], [srtlab netem cookbook].
- Realism features: **delay correlation** (e.g. 25% — each packet's delay correlates to the previous), and the **Gilbert-Elliott model** for **bursty loss** (more realistic for wireless than uniform random loss). Combine with **HTB** to impair only selected traffic classes / rate-limit. [oneuptime netem], [man7], [samwho.dev emulating bad networks].

5.3 iperf3 — throughput / loss / jitter bars

- Client-server tool measuring **TCP/UDP throughput, packet loss, jitter**. Control channel on **TCP 5201**; data on the same TCP/UDP port. Run server in one ns, client in another, measure tunnel goodput vs bare-link, and re-run under each netem profile to produce throughput-vs-impairment bars. [redhat iperf3 blog], [andrewbaker iperf3], [cisconetsolutions iperf3].

5.4 nftables — DPI simulation + kill-switch + leak blocking

- **Kill-switch (fwmark routing).** WireGuard tags tunnel traffic with an **fwmark**; an OUTPUT firewall rule **rejects any traffic NOT matching the WireGuard fwmark**, so if the tunnel drops, all egress fails closed. Classic WireGuard `PostUp` pattern (iptables/nftables). [ivpn WireGuard kill switch], [ivpn nftables leak prevention].
- **DNS-leak blocking (DPI-style port drop).** nftables rules that **drop UDP/53 and TCP/53 on the physical (non-VPN) interface** physically prevent any DNS query from escaping the tunnel — "brutal but effective"; leaking apps get connection-refused instead of reaching the ISP resolver. [WebSearch DNS-leak summary], [vpnsmith WireGuard DNS leak].
- A "DPI simulation" rig generalizes this: nftables (or netem corruption) on the middle namespace to **drop/mangle the VPN's wire signature** and confirm the obfuscation/transport still connects (fail-closed vs fall-through behaviour).

5.5 DNS / kill-switch / IPv6 leak testing

- **DNS leak test**: confirm DNS queries resolve via the tunnel resolver, not the local/ISP resolver (dnsleaktest.com-style, or query a controlled resolver in the upstream ns and assert the source). [WebSearch DNS-leak summary], [vpn.how 2026 leak testing].
- **Leak-test the full matrix**: **DNS, WebRTC, IPv6**. IPv6 is a common gap — either route v6 through the tunnel or **block v6 egress** at nftables; assert no v6 escapes. [vpn.how 2026], [a.vpn.how IPv6 leaks 2026].
- **Kill-switch test**: bring the tunnel interface down mid-iperf3/curl and assert **zero packets** reach the upstream ns (fail-closed), then restore. [ivpn kill switch], [WebSearch DNS-leak summary].
- **Priority order** (defense-in-depth): tunnel DNS config (systemd-resolved PostUp) → nftables drop of 53 outside tunnel → server-side resolver (Unbound/AdGuard Home) → server-side DNAT of UDP/53. [WebSearch DNS-leak summary].

Negative findings / gaps

- maybenot GitHub README does not print an explicit version banner; version 2.2.2 (core) confirmed via libraries.io, not the README. `maybenot-machines` 1.0.1 is from a search-result snippet, not a fetched crates.io page — treat the machines-crate version as **UNCONFIRMED** pending a direct crates.io read.
- Exact DAITA v2 padding-budget numbers and the relay defense-database size are not published by Mullvad in detail (the "half the

dummy packets" / "half average bandwidth overhead" figure is the public quantitative claim).
- DAITA v1 client machine numbers ([1.5,9.5]s, 3rd/5th-packet ratios) come from Tobias Pulls' (maybenot author) blog, which is authoritative for this design; Mullvad's own blog states the mechanisms qualitatively, not these constants.

Sources verified

- https://github.com/maybenot-io/maybenot — 2026-06-25
- https://arxiv.org/abs/2304.09510 (Maybenot: A Framework for Traffic Analysis Defenses) — 2026-06-25
- https://dl.acm.org/doi/10.1145/3603216.3624953 (WPES'23) — 2026-06-25
- https://libraries.io/cargo/maybenot (core v2.2.2) — 2026-06-25
- https://crates.io/crates/maybenot-simulator (v2.2.1) — 2026-06-25
- https://crates.io/crates/maybenot — 2026-06-25
- https://pulls.name/blog/2025-03-27-daita-v1-and-v2-defenses/ — 2026-06-25
- https://mullvad.net/en/vpn/daita — 2026-06-25
- https://mullvad.net/en/blog/daita-defense-against-ai-guided-traffic-analysis — 2026-06-25
- https://mullvad.net/en/blog/introducing-defense-against-ai-guided-traffic-analysis-daita — 2026-06-25
- https://mullvad.net/en/blog/defense-against-ai-guided-traffic-analysis-daita-now-available-on-linux-and-macos — 2026-06-25
- https://cyberinsider.com/mullvads-daita-v2-brings-stronger-resistance-to-ai-enhanced-vpn-traffic-analysis/ — 2026-06-25
- https://github.com/netbirdio/netbird/issues/2366 — 2026-06-25
- https://man7.org/linux/man-pages/man8/tc-netem.8.html — 2026-06-25
- https://oneuptime.com/blog/post/2026-03-04-simulate-network-latency-packet-loss-tc-netem-rhel-9/view — 2026-06-25
- https://oneuptime.com/blog/post/2026-03-02-how-to-use-ip-netns-for-network-namespace-testing-on-ubuntu/view — 2026-06-25
- https://girondi.net/post/network_namespaces/ — 2026-06-25
- https://srtlab.github.io/srt-cookbook/how-to-articles/using-netem-to-emulate-networks.html — 2026-06-25
- https://samwho.dev/blog/emulating-bad-networks/ — 2026-06-25
- https://www.redhat.com/en/blog/network-testing-iperf3 — 2026-06-25
- https://andrewbaker.ninja/2026/01/04/iperf3-the-engineers-swiss-army-knife-for-network-performance-testing/ — 2026-06-25
- https://www.cisconetsolutions.com/iperf-network-testing-and-troubleshooting-tool/ — 2026-06-25
- https://www.ivpn.net/knowledgebase/linux/linux-wireguard-kill-switch/ — 2026-06-25

- https://www.ivpn.net/knowledgebase/linux/linux-how-do-i-prevent-vpn-leaks-using-nftables-and-openvpn/ — 2026-06-25
- https://www.vpnsmith.com/en/blog/wireguard-dns-leak-prevention-2026 — 2026-06-25
- https://vpn.how/en/pages/vpn-leak-testing-in-2026-step-by-step-guide-with-dns-webrtc-and-ipv6-checks.html — 2026-06-25
- https://a.vpn.how/en/pages/ipv6-leaks-and-vpns-in-2026-how-to-seal-every-gap-without-losing-speed.html — 2026-06-25